

# LiteQL — Complete Project Guide

Oct 9, 2026 · @Shivaansh

LiteQL is a small compiler for a SQL-like language: it checks and plans a query against a CSV or JSON file before running anything, so mistakes are caught before any data row is read.

## 1. The big idea

You type one line such as `SELECT name, age FROM "students.csv" WHERE age > 20 ORDER BY age DESC LIMIT 5`. LiteQL does not run that text directly. It treats it like a program: it splits it into tokens, checks the grammar, checks it against the real file (do the columns exist? do the types fit?), converts it into a simple plan, improves the plan, and only then produces rows. If any early step fails, you get a clear error naming the phase that caught it, and the file's rows are never touched.

The project's claim is educational, not competitive. LiteQL is not faster than DuckDB and is not a database. It shows, in about 1,400 lines of readable Python, how the classical compiler phases apply to a small, realistic problem.

**Why this counts as a Compiler Design project** — every phase from the course appears, each in its own module:

| Course concept    | What it is in LiteQL                                              | Module                                               |
|-------------------|-------------------------------------------------------------------|------------------------------------------------------|
| Lexical analysis  | Query text becomes labelled tokens                                | <code>lexer.py</code> , <code>tokens.py</code>       |
| Syntax analysis   | Recursive-descent parser builds an AST from the grammar           | <code>parser.py</code> , <code>ast.py</code>         |
| Symbol table      | Column name to inferred type (NUMBER or TEXT), read from the file | <code>schema.py</code> , <code>data_source.py</code> |
| Semantic analysis | Do the named columns exist? Is the LIMIT valid?                   | <code>semantic.py</code>                             |
| Type checking     | Both sides of every comparison must have the same type            | <code>semantic.py</code>                             |

| Course concept              | What it is in LiteQL                                 | Module                       |
|-----------------------------|------------------------------------------------------|------------------------------|
| Intermediate representation | The logical plan: Scan, Filter, Project, Sort, Limit | <code>logical_plan.py</code> |
| Optimization                | Predicate pushdown and projection pushdown           | <code>optimizer.py</code>    |
| Code execution              | An interpreter that runs the plan and returns rows   | <code>executor.py</code>     |
| Error handling              | One error class per phase, with position markers     | <code>errors.py</code>       |

## The pipeline in one line

Query text → Lexer → Tokens → Parser → AST → Semantic analysis (symbol table + type check) → Logical plan → Optimizer → Optimized plan → Executor → Result

**Authors and course:** Shivaansh Aggarwal and Devansh Sharma, BCSE307L Compiler Design, faculty Dr. R Banupriya. Implementation language: Python 3 using only the standard library ( `pytest` is needed only for the tests ).

## 2. Problem, objectives and scope

The problem: filtering or sorting a CSV/JSON file normally means learning a data library or writing loops, and the usual mistakes only surface at run time.

### The mistakes LiteQL catches early

- Referring to a column that does not exist ( `score` in a file with `name, age, city` ).
- Comparing incompatible types (the number column `age` against the text `"hello"` ).
- A malformed query (missing `FROM` , a dangling `LIMIT` ).

### Objectives

1. Define a small SQL-like language with a written grammar.
2. Build every compiler phase explicitly: lexer, parser, AST, semantic analyzer with symbol table and type checking, logical-plan IR, optimizer, executor.
3. Make each phase's output visible from the command line so it can be demonstrated.
4. Prove the optimizer never changes a query's result.

**Scope.** The restricted scope is deliberate: it keeps the project finishable and lets every phase be shown clearly.

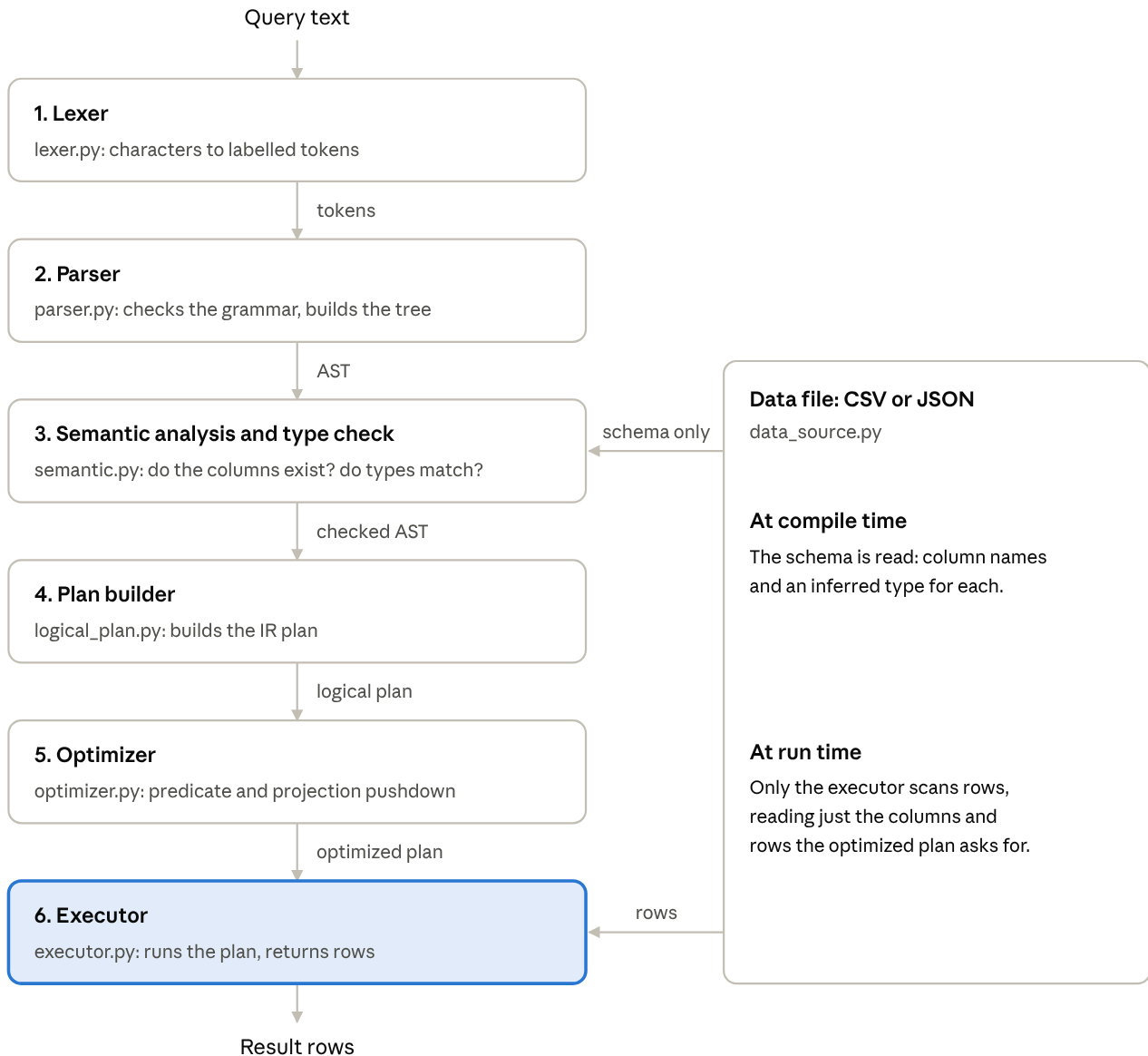
| Supported                                                                                                                                       | Intentionally not supported                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| One CSV or JSON file per query                                                                                                                  | Joins, multiple files, subqueries, CTEs                                                                                                 |
| <code>SELECT</code> with column names or <code>*</code>                                                                                         | Aggregates ( <code>COUNT</code> , <code>SUM</code> , <code>AVG</code> ), <code>GROUP BY</code> , <code>HAVING</code> , window functions |
| <code>FROM</code> , <code>WHERE</code> , <code>ORDER BY</code> ( <code>ASC</code> / <code>DESC</code> ), <code>LIMIT</code>                     | Arithmetic, string functions, parentheses, <code>LIKE</code> , <code>IN</code> , <code>NOT</code>                                       |
| <code>AND</code> , <code>OR</code> and <code>=</code> <code>!=</code> <code>&gt;</code> <code>&lt;</code> <code>&gt;=</code> <code>&lt;=</code> | <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , transactions, indexes                                                 |
| Column-to-value and column-to-column comparisons                                                                                                | Dates, booleans, <code>NULL</code> literals                                                                                             |
| <code>NUMBER</code> and <code>TEXT</code> types                                                                                                 | SQL compatibility in general                                                                                                            |
| Predicate pushdown and projection pushdown                                                                                                      | Any other optimization                                                                                                                  |

The parser names common unsupported words in its error message, for example `JOIN` is not supported by LiteQL .

### 3. Architecture

The pipeline is a straight line of six phases, each consuming the previous phase's output, and only the last one reads data rows.

## Only the executor consumes data rows



LiteQL pipeline · 6 phases, 1 data file

Read the diagram top to bottom. Arrow labels are the intermediate representations; each box names its module. The data file is touched twice: at compile time to learn the schema (the semantic phase needs to know the columns and types), and at run time by the executor.

## Module responsibilities

| Module                                            | Responsibility                                                                   | May read the data file?                          |
|---------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------|
| <code>tokens.py</code> ,<br><code>lexer.py</code> | Characters to tokens                                                             | No                                               |
| <code>ast.py</code> , <code>parser.py</code>      | Tokens to tree; syntax errors                                                    | No                                               |
| <code>schema.py</code>                            | <code>DataType</code> , <code>Schema</code> (the symbol table), type inference   | No (it is handed values)                         |
| <code>data_source.py</code>                       | CSV and JSON reading, typing of cells, scan with projection and predicate        | Yes, the only module that opens data files       |
| <code>semantic.py</code>                          | Column existence, type compatibility, LIMIT                                      | Only through the <code>Schema</code>             |
| <code>logical_plan.py</code>                      | Plan node classes; checked AST to plan                                           | No                                               |
| <code>optimizer.py</code>                         | Predicate and projection pushdown                                                | No                                               |
| <code>executor.py</code>                          | Runs plan nodes, comparison rules, statistics                                    | Yes, the only phase that scans rows              |
| <code>compiler.py</code>                          | Chains the phases, keeps every intermediate result in a <code>Compilation</code> | Via <code>data_source.py</code> , for the schema |
| <code>printer.py</code> ,<br><code>cli.py</code>  | Formatting and the command line                                                  | No                                               |

### Design rules that keep it explainable

- **One phase, one module.** Each phase takes the previous phase's output and nothing else, so each can be tested and shown alone.
- **Immutable intermediate forms.** AST and plan nodes are frozen, so the plan before optimization is still there to print afterwards.
- **File logic is isolated.** Swapping CSV or JSON handling never touches the lexer, parser or optimizer.

- **Compilation is separate from execution.** `compile_query` does everything up to the optimized plan; `Compilation.run()` is the only thing that executes. A test patches the scan to fail and proves compilation never reaches it.
- **Partial results survive errors.** When a phase fails, the exception carries the stages that finished, so the CLI can still show, for example, the tokens of a query that failed to parse.

## 4. One query, end to end

The main example query becomes five different representations before it returns five rows. Everything below is the program's real output ( `--all` ) for `examples/students.csv`.

```
SELECT name, age FROM "students.csv" WHERE age > 20 ORDER BY age DESC LIMIT 5
```

**Step 1 — Lexer: text to tokens.** Characters are grouped and labelled. Nothing is checked yet; the lexer does not know whether `age` exists.

```
SELECT IDENTIFIER(name) COMMA(,) IDENTIFIER(age) FROM
STRING(students.csv)
WHERE IDENTIFIER(age) GREATER(>) NUMBER(20)
ORDER BY IDENTIFIER(age) DESC LIMIT NUMBER(5) EOF
```

**Step 2 — Parser: tokens to AST.** The parser checks the tokens follow the grammar and builds a tree. Commas and keywords are gone; only structure remains.

```
Query
├─ Select: name, age
├─ From: students.csv
├─ Where
│   └─ age > 20
├─ Order By: age DESC
└─ Limit: 5
```

**Step 3 — Symbol table.** The file is opened to learn its columns and infer each type from its values.

```
name -> TEXT
age  -> NUMBER
city -> TEXT
```

**Step 4 — Semantic analysis and type checking.** `name` and `age` exist; `age` is NUMBER and `20` is a number, so `age > 20` is valid. Nothing is printed on success; this step produces the checked AST (column names rewritten to the file's own spelling).

**Step 5 — Logical plan (the intermediate representation).** Read it bottom-up: scan the file, keep rows with `age > 20`, keep two columns, sort, cut to five.

```
Limit 5
  Sort age DESC
    Project name, age
      Filter age > 20
        Scan students.csv
```

**Step 6 — Optimizer.** The `Filter` is merged into the `Scan` (predicate pushdown) and the `Scan` is told to read only `name`, `age` (projection pushdown). The result is the same; less data flows upward.

```
Limit 5
  Sort age DESC
    Project name, age
      Scan students.csv (predicate: age > 20; columns: name, age)
```

**Step 7 — Executor: plan to rows.**

```
name    age
-----  ---
Vihaan  31
Meera   28
Arjun   26
Diya    24
Kabir   22
(5 rows)
```

The work saved is measured, not claimed. With `--stats` the Scan reports 8 rows and 24 fields passed upward without optimization, and 6 rows and 12 fields with it. The file is read in full either way, and the two results are verified identical.

## 5. Each phase in depth

The phases run in a fixed order, and each one only uses what the previous one produced. The first four (below) are the front end: they never execute anything.

## 5.1 Lexer — `lexer.py`, `tokens.py`

The lexer scans the query once, left to right, and emits a list of `Token(type, text, position)` ending with `EOF`. It uses regular expressions for names, numbers and file names, and a short operator table.

- **Keywords** `SELECT FROM WHERE ORDER BY ASC DESC LIMIT AND OR` are case-insensitive; the original spelling is kept in the token text. A word that merely starts like a keyword ( `selection` ) is an ordinary identifier.
- **Strings** use `'...'` or `"..."`, with no escape sequences. Quoted file names are strings too.
- **Bare file names** such as `titanic.csv` or `data/people.json` become a `FILENAME` token, only valid after `FROM`.
- **Numbers:** `20`, `3.5`, `-4`. A number glued to letters ( `20abc` ) is a lexical error rather than two tokens.
- **Operators and punctuation:** `= != > < >= <= , * ;`. Two-character operators are matched first so `>=` is never read as `>` then `=`.
- **Lexical errors:** unexpected character ( `@`, `#`, `(` ), unterminated string, malformed number. Each reports its position; parentheses get the hint that LiteQL has none.

The lexer never checks meaning: `SELECT zzz FROM 'nowhere.csv'` tokenizes without complaint.

## 5.2 Parser — `parser.py`

A hand-written recursive-descent parser with one method per grammar rule, so the code reads side by side with `docs/grammar.md`: `parse_query`, `parse_column_list`, `parse_condition`, `parse_and_term`, `parse_comparison`, `parse_value`, `parse_order_clause`, `parse_limit_clause`. It looks at the current token, consumes what the rule allows, and fails with a message that says what was expected and what was found.

- **Precedence.** `AND` binds tighter than `OR`, as in SQL: `a OR b AND c` means `a OR (b AND c)`. This is done by splitting the documented `condition` rule into `condition` (`OR` level) and `and_term` (`AND` level); the accepted language is unchanged.
- **Helpful errors.** Out-of-order clauses ( `LIMIT 5 ORDER BY age` ) say the clauses must appear as `WHERE`, `ORDER BY`, `LIMIT`. Words like `JOIN`, `GROUP`, `LIKE`, `COUNT` are named as unsupported.

- **Safety limit.** A WHERE clause may hold at most 100 comparisons, so a pathological query cannot overflow Python's recursion limit.
- **It never opens the data file.** Parsing `FROM "does/not/exist.csv"` succeeds; the file is only looked at afterwards.

### 5.3 AST — `ast.py`, printed by `printer.py`

The AST holds structure, not tokens. All nodes are frozen dataclasses, so no later phase can silently change them.

- `Query(columns, source, where, order_by, limit)` where `columns=None` means `SELECT *`.
- `Column`, `OrderBy(column, direction)`.
- Values: `StringLiteral`, `NumberLiteral`, `ColumnReference`.
- Conditions: `Comparison(column, operator, value)`, `And(left, right)`, `Or(left, right)` — a binary tree, so precedence is visible in the shape.

Example: `WHERE a = 1 OR b = 'x' AND c >= 2` prints as

```

└─ Where
  └─ OR
    └─ a = 1
      └─ AND
        └─ b = 'x'
          └─ c >= 2

```

### 5.4 Schema inference and the symbol table — `schema.py`, `data_source.py`

All file-specific code lives behind a small `DataSource` class (`CSVDataSource`, `JSONDataSource`). It can report the schema, and it can scan rows reading only requested columns and dropping rows that fail a predicate — the two hooks the optimizer uses.

The symbol table is a `Schema`: an ordered map from column name to `DataType.NUMBER` or `DataType.TEXT`. A column is NUMBER only if **every** non-blank value in the file is a number; one stray text value makes it TEXT, and an all-blank column is TEXT. Inference reads all rows (not a sample) so a late text value cannot make a NUMBER column crash.

- **CSV:** parsed with Python's `csv` module (quoted commas work, a byte-order mark is stripped, blank lines skipped, short rows leave missing values). Cells are text, so `22`, `-2.5`, `.5` and `1e3` count as numbers; `nan`, `1,5` and `12abc` do not.

- **JSON:** a top-level array of objects. Column names are the union of keys in first-seen order; absent keys and `null` are missing values. The string `"20"` stays TEXT; booleans and nested values become TEXT.

## 5.5 Semantic analysis and type checking — `semantic.py`

A query can be grammatical and still meaningless. `SELECT salary FROM "students.csv"` parses perfectly; only the symbol table reveals that `salary` is not a column. This is the difference between a syntax error (wrong shape) and a semantic error (wrong meaning), and it is the phase that turns LiteQL from a parser into a checker.

Checks, in the order the clauses appear:

1. **Column existence** for the SELECT list, every column in WHERE (both sides of a column-to-column comparison), and ORDER BY. Names match case-insensitively; an exact match wins. The AST is rewritten with the file's own spelling, so `age` becomes `Age` if that is the header.
2. **No duplicate** selected columns.
3. **Type compatibility** of every comparison: NUMBER with NUMBER, TEXT with TEXT. A number literal is NUMBER, a quoted value is TEXT, a column has its inferred type.
4. **LIMIT** must not be negative.

Messages name both sides and the phase:

```
Semantic Error: Column 'salary' does not exist.
  Available columns: name, age, city
```

```
Type Error: Cannot compare NUMBER column 'age' with TEXT value 'hello'
```

If the missing name is on the right of a comparison ( `sex = M` ), the message adds a hint that text values need quotes. TEXT columns may use `>`, `<`, `>=`, `<=` (plain, case-sensitive character order); only mixing NUMBER and TEXT is a type error. This phase reads no data rows.

## 5.6 Logical plan — `logical_plan.py`

The plan is the intermediate representation: five immutable node classes, each (except `Scan`) holding one `child`.

| Node    | Meaning                                     | Fields                                                         |
|---------|---------------------------------------------|----------------------------------------------------------------|
| Scan    | Read the file                               | source , columns (None = all),<br>predicate (None = no filter) |
| Filter  | Keep rows that satisfy a condition          | child , condition                                              |
| Project | Keep only the listed columns, in that order | child , columns                                                |
| Sort    | Order rows by one column                    | child , column , direction                                     |
| Limit   | Keep the first N rows                       | child , count                                                  |

`build_plan` stacks them in the order Scan, Filter, Project, Sort, Limit, skipping any the query does not use ( `SELECT *` has no Project). One subtlety: if `ORDER BY` names a column that is not selected, Sort is placed *below* Project; otherwise Project would discard the column before Sort could use it. Because nodes are immutable, the original plan survives optimization and both can be printed.

## 5.7 Optimizer — `optimizer.py`

Exactly two rewrites, applied in this order. Neither may change the result.

**Predicate pushdown.** A `Filter` sitting directly on a `Scan` is merged into it as the scan's `predicate`, so non-matching rows are rejected as they are read instead of being passed up and discarded. The rewrite is deliberately narrow: a filter is never moved below `Limit`, because that would change which rows the limit keeps.

**Projection pushdown.** The `Scan` is told which columns the whole plan needs: the `SELECT` list, then the `WHERE` columns, then the `ORDER BY` column, without duplicates. So `SELECT name FROM ... WHERE age > 20` still reads `age`. For `SELECT *` there is no Project node, so every column is needed and nothing is removed. This was a real bug in the earlier prototype (a `SELECT *` query returned only the filter column), and it now has a regression test.

The optimizer also returns short notes ( `Predicate pushdown: ...`, `Projection pushdown: Scan reads only name, age` ) that the CLI prints under the optimized plan. No speed-up percentage is claimed anywhere; `--stats` counts rows and fields leaving the Scan for both plans and confirms the results match.

## 5.8 Executor — `executor.py`

A small recursive interpreter: to run a node, run its child, then apply the node's operator. It is the only phase that consumes data rows.

- **Scan** opens the data source and yields typed rows, applying `columns` and `predicate` if present.
- **Filter** evaluates the condition per row. **Project** builds a dict per row in the SELECT order. **Limit** slices.
- **Sort** is stable, so ties keep file order in both directions.
- **Missing values**: any comparison involving a missing value is false (even `!=`), and ORDER BY puts missing values last in both directions. They print as `NULL`.
- `ExecutionStats` records rows read, rows and fields passed up by the Scan, and rows returned.

## 5.9 Errors — `errors.py`

| Class                       | Raised by             | Example message                                                         | CLI exit code |
|-----------------------------|-----------------------|-------------------------------------------------------------------------|---------------|
| <code>LexicalError</code>   | lexer                 | Unexpected character '@' at position 7                                  | 2             |
| <code>SyntaxError</code>    | parser                | Expected FROM after SELECT column list, but found string 'students.csv' | 2             |
| <code>SemanticError</code>  | semantic analysis     | Column 'salary' does not exist.                                         | 2             |
| <code>TypeError</code>      | semantic analysis     | Cannot compare NUMBER column 'age' with TEXT value 'hello'              | 2             |
| <code>ExecutionError</code> | file lookup / reading | Unable to read file 'missing.csv' (file not found)                      | 2             |

All subclass `LiteQLError`, and the message always starts with the phase name. Lexical and syntax errors also print the query with a `^` under the problem. `SyntaxError` and `TypeError` reuse the names from the project specification, so modules import them

through `errors` and never shadow Python's built-ins. When a phase fails, the error carries a partial `Compilation`, which is how `--tokens` or `--schema` can still show what finished before the failure.

## 6. Data handling rules

Real files are messy, so LiteQL fixes a small set of rules and applies them everywhere; this table is the quick reference to answer "what happens if...".

| Situation                                                                                      | What LiteQL does                                                                                                     |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Blank CSV cell, short CSV row, absent JSON key, JSON <code>null</code>                         | Treated as a missing value, shown as <code>NULL</code>                                                               |
| Comparison with a missing value ( <code>=</code> , <code>!=</code> , <code>&gt;</code> , ... ) | Always false, so the row is filtered out                                                                             |
| <code>ORDER BY</code> with missing values                                                      | They come last, in both ASC and DESC                                                                                 |
| Column with one non-numeric value                                                              | Whole column is TEXT                                                                                                 |
| All-blank column                                                                               | TEXT                                                                                                                 |
| Column names                                                                                   | Case-insensitive ( <code>age</code> finds <code>Age</code> ); exact match preferred; output uses the file's spelling |
| String values                                                                                  | Case-sensitive ( <code>'male'</code> does not match <code>'Male'</code> )                                            |
| Text ordering ( <code>&gt;</code> , <code>&lt;</code> ) on TEXT columns                        | Allowed, plain character order                                                                                       |
| Number vs text ( <code>age &gt; 'x'</code> , <code>city = 20</code> )                          | Type Error                                                                                                           |
| Unquoted word as a value ( <code>sex = M</code> )                                              | Read as a column name; if it does not exist, Semantic Error with a hint to add quotes                                |
| JSON string <code>"20"</code>                                                                  | TEXT (it is a string in JSON)                                                                                        |
| JSON booleans / nested values                                                                  | TEXT ( <code>true</code> , <code>false</code> , or JSON text)                                                        |
| CSV with a byte-order mark, quoted commas, blank lines                                         | Handled                                                                                                              |

| Situation                                   | What LiteQL does                                               |
|---------------------------------------------|----------------------------------------------------------------|
| Header-only CSV or empty JSON array         | Valid; zero rows (JSON <code>[]</code> also has no columns)    |
| Unsupported extension ( <code>.txt</code> ) | Execution Error: only <code>.csv</code> and <code>.json</code> |
| Sort ties                                   | Keep file order (stable sort)                                  |
| <code>LIMIT 0</code>                        | Valid, returns zero rows                                       |
| <code>LIMIT -1</code>                       | Semantic Error                                                 |

**How `FROM` finds the file.** The name after `FROM` decides which file is read, quoted or bare. It is looked up in this order:

1. If `--file` has the same file name, that path is used.
2. The name as written, relative to the current directory.
3. The folder of the `--file` argument.
4. The project's `examples/` folder.

If none exists the error is `Execution Error: Unable to read file 'x.csv' (file not found)`. `--file` is a lookup hint, not an override, so a query can never silently read a different file from the one it names.

## 7. Install and run

LiteQL needs Python 3.9 or newer and nothing else; the suite was run on 3.9, and the demo was also run on 3.11 and 3.13. Run every command from the project root ( `LiteQL/` ).

### Install

```
python3 -m venv .venv && source .venv/bin/activate    # optional
python3 -m pip install -r requirements.txt            # installs pytest, used
only for tests
```

### Run a query

```
python3 -m liteql.cli --file examples/students.csv --query 'SELECT name, age
FROM "students.csv" WHERE age > 20 ORDER BY age DESC LIMIT 5'
```

`python3 -m liteql ...` works too. The query can also use a bare file name and be typed without `--file`:

```
python3 -m liteql.cli --query "SELECT * FROM titanic.csv WHERE age > 20 AND
sex = 'male' order by age desc limit 10"
```

## Options

| Flag                          | What it does                                                                                            |
|-------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>--query Q</code>        | The LiteQL query (required unless <code>--demo</code> )                                                 |
| <code>--file F</code>         | Data file; also the folder where <code>FROM</code> names are looked up                                  |
| <code>--tokens</code>         | Show the token stream                                                                                   |
| <code>--ast</code>            | Show the abstract syntax tree                                                                           |
| <code>--schema</code>         | Show the symbol table                                                                                   |
| <code>--plan</code>           | Show the unoptimized logical plan                                                                       |
| <code>--optimized-plan</code> | Show the optimized plan plus the optimizer's notes                                                      |
| <code>--stats</code>          | Table of rows/fields passed up by the Scan, unoptimized vs optimized, and whether results are identical |
| <code>--all</code>            | All of the above                                                                                        |
| <code>--demo</code>           | The complete guided demonstration                                                                       |

With no stage flag, only the result table is printed. Stage flags add sections labelled `Tokens:`, `AST:`, `Symbol Table:`, `Logical Plan:`, `Optimized Plan:`, `Work done by the Scan:`, then `Result:`.

**Exit codes.** `0` success; `2` any LiteQL error (printed to standard error, with a `^` marker for lexical and syntax errors) or a usage error.

## Use it as a library

```

from liteql import compile_query

compilation = compile_query('SELECT name FROM "students.csv" WHERE age > 20',
                             "examples/students.csv")
rows, stats = compilation.run()                                # optimized plan;
run(optimized=False) for the plain one
print(compilation.tokens, compilation.ast, compilation.schema,
      compilation.plan, compilation.optimized_plan)

```

## Run the tests

```
python3 -m pytest
```

339 tests, about 7 seconds.

## 8. Demo script for the teacher

`python3 -m liteql.cli --demo` runs all five demonstrations in one go (about 190 lines of output). For a live walk-through, use the individual commands below so you can pause and explain. All use the main query on `examples/students.csv`.

### Demo 1 — a valid query through every phase

```
python3 -m liteql.cli --file examples/students.csv --all --query 'SELECT
name, age FROM "students.csv" WHERE age > 20 ORDER BY age DESC LIMIT 5'
```

Say: tokens come from the lexer, the AST from the parser, the symbol table from the file, then the logical plan, the optimized plan, and finally rows. Expected rows: Vihaan 31, Meera 28, Arjun 26, Diya 24, Kabir 22.

### Demo 2 — syntax error, caught by the parser

```
python3 -m liteql.cli --file examples/students.csv --tokens --query 'SELECT
name, age "students.csv"'
```

Say: lexing succeeded (the tokens are printed), but the structure is wrong because `FROM` is missing. A `^` marks the spot, and no data row is read. Expect: `Syntax Error: Expected FROM after SELECT column list, but found string 'students.csv' at position 17.`

### Demo 3 — missing column, caught by semantic analysis

```
python3 -m liteql.cli --file examples/students.csv --schema --query 'SELECT
name, salary FROM "students.csv"'
```

Say: this query is grammatical, so the parser accepts it; the symbol table ( name, age, city ) shows salary does not exist. Expect Semantic Error: Column 'salary' does not exist. with the available columns listed. No plan is built.

#### Demo 4 — type error

```
python3 -m liteql.cli --file examples/students.csv --schema --query 'SELECT
name FROM "students.csv" WHERE age > "hello"'
```

Say: age is NUMBER in the symbol table and "hello" is TEXT. Expect Type Error: Cannot compare NUMBER column 'age' with TEXT value 'hello'.

#### Demo 5 — optimization, before and after

```
python3 -m liteql.cli --file examples/students.csv --plan --optimized-plan --
stats --query 'SELECT name, age FROM "students.csv" WHERE age > 20 ORDER BY
age DESC LIMIT 5'
```

Say: in the first plan the Filter sits above the Scan; in the second it is gone and appears as the Scan's predicate. The Scan also reads only name, age. Point out that age stays because WHERE and ORDER BY need it. The stats table shows 8 rows and 24 fields leaving the Scan before, 6 rows and 12 after, and Results identical: yes. Say plainly that you report counts, not a speed-up percentage.

#### Bonus — queries the teacher may type

```
python3 -m liteql.cli --query "SELECT * FROM titanic.csv WHERE age > 20 AND
sex = 'male' order by age desc limit 10"
python3 -m liteql.cli --file examples/students.json --query 'SELECT name FROM
"students.json" WHERE age >= 28 ORDER BY age'
python3 -m liteql.cli --query "SELECT Name, Age FROM titanic.csv WHERE Pclass
= 1 OR Fare > 50 ORDER BY Age" --optimized-plan
python3 -m liteql.cli --query "SELECT * FROM titanic.csv WHERE sex = M"
```

Things to know before you present:

- examples/titanic.csv is a small **synthetic** file with Titanic-style columns, blank ages and names containing commas. Its Sex values are male and female, so sex = 'M' is valid but returns (0 rows).

- Lowercase keywords, lowercase column names and a trailing `;` all work.
- `examples/valid_queries.txt` and `examples/invalid_queries.txt` hold ready-made queries; each invalid one is tagged with the phase that must reject it.

## 9. Testing and correctness evidence

The suite has 339 pytest tests, all passing on Python 3.9.6, built to check behaviour and not just to raise the count.

| Test file                       | Tests | Covers                                                                                                      |
|---------------------------------|-------|-------------------------------------------------------------------------------------------------------------|
| <code>test_lexer.py</code>      | 47    | Keywords, identifiers, numbers, strings, operators, file names, positions, lexical errors                   |
| <code>test_parser.py</code>     | 57    | Valid forms, precedence, 28 malformed queries with expected messages, unsupported-word hints                |
| <code>test_ast.py</code>        | 7     | Exact AST structure, immutability, tree printer layout                                                      |
| <code>test_schema.py</code>     | 32    | CSV and JSON schema, inference rules, messy files, file lookup, read errors                                 |
| <code>test_semantic.py</code>   | 22    | Missing columns, type errors, case rules, no data read during checking                                      |
| <code>test_plan.py</code>       | 8     | AST to plan shape, Sort placement, immutability                                                             |
| <code>test_optimizer.py</code>  | 30    | Both pushdowns, required columns, 700 generated queries optimized = unoptimized                             |
| <code>test_executor.py</code>   | 30    | Each operator, all six comparisons, AND/OR precedence, missing values, stable sort, JSON                    |
| <code>test_end_to_end.py</code> | 87    | Shipped example files, all five demos, Titanic-style queries, CLI flags and exit codes, <code>--demo</code> |
| <code>test_robustness.py</code> | 19    | Seeded fuzzing, odd input, the 100-comparison limit                                                         |

### Four independent lines of evidence

1. **Optimizer equivalence.** 700 generated combinations of select list, WHERE, ORDER BY and LIMIT on the Titanic-style file each give identical rows with and without optimization.
2. **An independent oracle.** 900 queries are compared against plain Python working directly on the raw CSV (no LiteQL code), including AND/OR precedence, missing values and sort order.
3. **Fuzzing.** About 8,600 random token sequences, corrupted valid queries and queries against deliberately messy CSV/JSON files. The only acceptable outcomes are rows or a clean `LiteQLError`; whenever rows come back, optimized must equal unoptimized.
4. **Mutation checks.** Fifteen bugs were deliberately re-introduced into a scratch copy (dropping WHERE columns from projection pushdown, disabling the type check, sorting missing values first, an off-by-one LIMIT, and so on). The suite failed for every one, after one extra test closed a gap in the first round.

### Structural guarantees the tests assert

- Compiling a query, valid or invalid, never scans a data row; only `Compilation.run()` does.
- Every invalid query in `examples/invalid_queries.txt` fails in its tagged phase.
- Every query in `examples/valid_queries.txt` runs and optimizes to the same rows.

### Bugs found and fixed in the earlier prototype (each now has a regression test)

- `SELECT *` with `WHERE` returned only the filter column, so the optimized result differed from the unoptimized one.
- Blank CSV values crashed with a Python `TypeError` (Titanic has blank ages).
- `ORDER BY` on an unselected column silently did not sort.
- Bare `FROM titanic.csv`, a trailing `;` and negative numbers were lexical errors.
- Column names were case-sensitive.
- `--file` silently overrode the file named in `FROM`.

**Not verified:** installing the package with `pip install -e .`, and running `pytest` on Python 3.11 and 3.13 (`pytest` is not installed there; only the demo and a query were run on those versions).

## 10. Design decisions and assumptions

Where the project documents were silent or ambiguous, the simplest consistent choice was made. These are the choices to be ready to defend, and the first four are the ones to confirm.

| # | Decision                                                                                                  | Why                                                                                       | Where to change it                                                                   |
|---|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 1 | <code>AND</code> binds tighter than <code>OR</code> (SQL rule)                                            | The grammar gives no precedence; users expect SQL behaviour                               | <code>parse_condition</code> / <code>parse_and_term</code> in <code>parser.py</code> |
| 2 | Bare file names ( <code>FROM titanic.csv</code> ) allowed besides quoted ones                             | The grammar says <code>FROM STRING</code> , but unquoted names are natural to type        | <code>FILENAME_RE</code> in <code>lexer.py</code> , <code>parse_source</code>        |
| 3 | <code>FROM</code> decides the file; <code>-file</code> only helps find it                                 | Prevents silently querying a different file than the query names                          | <code>resolve_path</code> in <code>data_source.py</code>                             |
| 4 | Column names are case-insensitive; string values are case-sensitive                                       | <code>age</code> should find a header <code>Age</code> ; exact value matches are expected | <code>Schema.resolve</code> in <code>schema.py</code>                                |
| 5 | Type inference reads all rows, not a sample                                                               | A late stray text value must not make a <code>NUMBER</code> column wrong                  | <code>infer_schema</code> in <code>schema.py</code>                                  |
| 6 | Missing values never match a comparison and sort last                                                     | Avoids crashes and gives one predictable rule                                             | <code>evaluate</code> and <code>Sort</code> in <code>executor.py</code>              |
| 7 | <code>TEXT</code> columns allow <code>&gt;</code> <code>&lt;</code> <code>&gt;=</code> <code>&lt;=</code> | Only mixing <code>NUMBER</code> and <code>TEXT</code> is incompatible                     | <code>_check_comparison</code> in <code>semantic.py</code>                           |
| 8 | Sort sits below Project when <code>ORDER BY</code> uses an unselected column                              | Otherwise the sort column is already gone                                                 | <code>build_plan</code> in <code>logical_plan.py</code>                              |
| 9 | Trailing <code>;</code> and negative numbers accepted                                                     | Common habits when typing queries                                                         | <code>lexer.py</code> , <code>parse_query</code>                                     |

| #  | Decision                                                                                                                              | Why                                                     | Where to change it                                     |
|----|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------------|
| 10 | A WHERE clause may hold at most 100 comparisons                                                                                       | Guards Python's recursion limit with a clear error      | <code>MAX_COMPARISONS</code> in <code>parser.py</code> |
| 11 | <code>DataType</code> lives in <code>schema.py</code> (no <code>types.py</code> ); <code>compiler.py</code> is an extra driver module | Only two types exist; the driver keeps phases decoupled | file layout                                            |

### Other assumptions

- Quoted strings have no escape sequences; to include a quote character, use the other quote style.
- A JSON source must be a top-level array of objects.
- `examples/titanic.csv` is synthetic and exists to exercise messy data; it is not the real Titanic dataset.
- The Team Guide (which includes the speaking script) and the project brief were used as the source of truth. The Final Report (.docx) and the Review 1 deck were not opened, so check their wording of the grammar and plan shapes against decisions 1 to 3.

## 11. Limitations and future extensions

LiteQL is a teaching compiler, and the limits below are the honest edge of what it does.

### Limitations and edge cases

- Everything in the "not supported" half of the scope table in section 2 is rejected, not partially handled.
- Column names that are keywords (`order`, `limit`, ...) or contain spaces or punctuation (such as `Siblings/Spouses Aboard`) cannot be named in a query. `SELECT *` still returns them.
- Quoted strings have no escape sequences. JSON nested values are shown as JSON text.
- The whole file is read into memory when it is opened, and again when the query executes. This suits demo-sized files, not large data.

- Pushdown reduces what flows between operators, which `--stats` reports. It does not make the file read lazier, because the data source loads the file first.
- Numeric-looking text such as a ZIP code `01234` in a CSV column is treated as a number.
- Text comparison and sorting are case-sensitive and use plain character order.
- The optimizer is a fixed two-rule pipeline over a one-table plan, not a cost-based optimizer.
- A WHERE clause is capped at 100 comparisons.

**Possible future extensions** (none are implemented, and none are claimed)

- Parentheses in conditions, `LIKE`, `IS NULL`, `NOT`.
- A date type.
- Streaming execution (row at a time), so pushdown also reduces what is read.
- A rule that merges `Sort` and `Limit` into a top-N step.
- Far beyond this project's scope: aggregates, `GROUP BY` and joins.

## 12. Literature context and research gaps

LiteQL's contribution is educational and architectural: a small, transparent pipeline where established compiler and query-optimization ideas can be watched working. It does not claim a new optimization algorithm.

The sources below are those named in the project brief, with the role each plays in the argument. DOIs are copied from the brief and were not independently checked here, and the papers themselves were not summarized or quoted.

| Source                                                                       | Venue     | Role in the project                                                        |
|------------------------------------------------------------------------------|-----------|----------------------------------------------------------------------------|
| Aho, Lam, Sethi, Ullman, <i>Compilers: Principles, Techniques, and Tools</i> | Textbook  | Foundation for the lexer, parser, semantic analysis, IR and optimization   |
| Levy, Mumick, Sagiv, <i>Query Optimization by Predicate Move-Around</i>      | VLDB 1994 | Foundational work on moving predicates, the idea behind predicate pushdown |

| Source                                                                     | Venue                                       | Role in the project                                                 |
|----------------------------------------------------------------------------|---------------------------------------------|---------------------------------------------------------------------|
| Yan, Lin, He, <i>Predicate Pushdown for Data Science Pipelines</i>         | SIGMOD 2023,<br>doi:10.1145/3589281         | Shows predicate pushdown is still relevant in modern data pipelines |
| Zhang, Campbell, Tang, Dillig, <i>Optimal Predicate Pushdown Synthesis</i> | PLDI 2026 (PACMPL),<br>doi:10.1145/3808312  | Recent work: predicate pushdown remains an active research problem  |
| Raasveldt, Mühleisen, <i>DuckDB: An Embeddable Analytical Database</i>     | SIGMOD 2019,<br>doi:10.1145/3299869.3320212 | A mature analytical engine as a comparison point                    |

### The three gaps LiteQL addresses

1. Production engines such as DuckDB are powerful but too large and complex to serve as a transparent, student-scale demonstration of every compiler stage.
2. Predicate pushdown is important and still researched, but it usually appears as an internal component of large systems, not as a small pipeline where the rewrite can be observed directly.
3. Semantic validation, type checking and schema-aware checking are normally bundled inside mature engines; LiteQL isolates them as explicit, inspectable stages.

### Safe and unsafe ways to describe the project

| Say                                                                   | Never say                                               |
|-----------------------------------------------------------------------|---------------------------------------------------------|
| LiteQL demonstrates predicate pushdown in a small compiler pipeline   | LiteQL is faster than DuckDB                            |
| LiteQL is a transparent implementation for studying compiler phases   | LiteQL is a new query optimization algorithm            |
| LiteQL applies established ideas to a constrained file-query language | LiteQL solves predicate pushdown, or replaces databases |

## 13. Viva questions and answers

Every answer points at something you can open and show in the code.

| Question                                      | Answer                                                                                                                                  | Show this                                                     |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Why is this a compiler project?               | The query is never run directly. It passes through lexing, parsing, semantic analysis, an IR, optimization and then execution.          | <code>compile_query</code> in <code>compiler.py</code>        |
| Why SQL-like?                                 | It is small, readable, and naturally needs every classical phase.                                                                       | <code>docs/grammar.md</code>                                  |
| What does the lexer do?                       | Turns characters into labelled tokens such as <code>IDENTIFIER(age)</code> . It never checks meaning.                                   | <code>lexer.py</code> , <code>--tokens</code>                 |
| Why a parser after the lexer?                 | The lexer gives a flat list. The parser checks the order against the grammar and builds a tree.                                         | <code>parser.py</code> , <code>--ast</code>                   |
| What is recursive descent?                    | One function per grammar rule, each calling the rules it contains.                                                                      | <code>parse_query</code> calling <code>parse_condition</code> |
| What is the AST?                              | A tree of the query's structure, with no commas or keywords left.                                                                       | <code>ast.py</code> , <code>format_ast</code>                 |
| How do you handle <code>a OR b AND c</code> ? | <code>AND</code> binds tighter, as in SQL: <code>a OR (b AND c)</code> . It is done with two grammar rules, and the AST shape shows it. | <code>parse_and_term</code>                                   |

| Question                         | Answer                                                                                                                                                | Show this                                                   |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| What is semantic analysis?       | Checking that a grammatical query makes sense for this file: columns exist, types fit, LIMIT is valid.                                                | <code>semantic.py</code>                                    |
| Syntax error vs semantic error?  | <code>SELECT name "x.csv"</code> has the wrong shape (syntax). <code>SELECT salary FROM ...</code> has the right shape but no such column (semantic). | Demos 2 and 3                                               |
| What is the symbol table?        | Column name to inferred type (NUMBER or TEXT), built from the file.                                                                                   | <code>Schema</code> , <code>--schema</code>                 |
| How is a column's type inferred? | NUMBER if every non-blank value is a number, otherwise TEXT.                                                                                          | <code>infer_schema</code>                                   |
| How does type checking work?     | For each comparison, the column's type must equal the other side's type.                                                                              | <code>_check_comparison</code>                              |
| What is the IR and why?          | The logical plan: five simple operators between the query and execution, easy to print, rewrite and run.                                              | <code>logical_plan.py</code> , <code>--plan</code>          |
| What is predicate pushdown?      | Moving the filter into the scan so unwanted rows are rejected while reading.                                                                          | <code>push_predicate</code> , <code>--optimized-plan</code> |

| Question                                                                           | Answer                                                                                                                      | Show this                                             |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| What is projection pushdown?                                                       | Reading only the columns the query needs: SELECT, WHERE and ORDER BY columns.                                               | <code>required_columns</code>                         |
| Why is <code>age</code> still read in <code>SELECT name WHERE age &gt; 20</code> ? | WHERE needs it, even though it is not returned. Dropping it would change the result.                                        | <code>test_where_column_is_kept...</code>             |
| How do you know the optimizer is safe?                                             | 700 generated queries and thousands of fuzzed ones give the same rows optimized and unoptimized.                            | <code>test_optimizer.py</code> , <code>--stats</code> |
| How much faster is it?                                                             | We do not claim a percentage. <code>--stats</code> counts rows and fields leaving the Scan; the file is still read in full. | <code>format_stats</code> in <code>cli.py</code>      |
| Why no joins?                                                                      | They enlarge the grammar and plan without adding a new compiler phase.                                                      | Scope table                                           |
| What happens to a missing value?                                                   | Comparisons with it are false and it sorts last.                                                                            | <code>evaluate</code> in <code>executor.py</code>     |
| Are invalid queries executed at all?                                               | No. Compilation never scans a row; a test enforces this.                                                                    | <code>test_semantic_phase_reads_no_data_rows</code>   |
| Why Python?                                                                        | Standard-library <code>csv</code> and <code>json</code> let the effort go into the compiler logic.                          | <code>data_source.py</code>                           |

| Question                  | Answer                                                                         | Show this  |
|---------------------------|--------------------------------------------------------------------------------|------------|
| Is it better than DuckDB? | No, and it does not claim to be. It is a small, transparent teaching pipeline. | Section 12 |

## 14. File-by-file reference

The source package is 1,398 lines across 16 files. Read them in the order of the table, which follows the pipeline.

| File                                | Lines | What it does                                                                                                                                                                                                                                |
|-------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>liteql/tokens.py</code>       | 58    | <code>TokenType</code> enum, <code>Token(type, text, position)</code> , token printing                                                                                                                                                      |
| <code>liteql/lexer.py</code>        | 100   | <code>tokenize()</code> : text to tokens; keyword table, operator table, file-name/number/identifier patterns                                                                                                                               |
| <code>liteql/ast.py</code>          | 76    | Frozen dataclasses: <code>Query</code> , <code>Column</code> , <code>Comparison</code> , <code>And</code> , <code>Or</code> , <code>OrderBy</code> , <code>StringLiteral</code> , <code>NumberLiteral</code> , <code>ColumnReference</code> |
| <code>liteql/parser.py</code>       | 189   | <code>Parser</code> class, one method per grammar rule; <code>parse()</code> ; error wording; the 100-comparison limit                                                                                                                      |
| <code>liteql/schema.py</code>       | 69    | <code>DataType</code> , <code>Schema</code> (the symbol table, with case-insensitive <code>resolve</code> ), <code>infer_schema</code>                                                                                                      |
| <code>liteql/data_source.py</code>  | 146   | <code>CSVDataSource</code> , <code>JSONDataSource</code> , <code>typed_scan(columns, predicate, stats)</code> , <code>resolve_path</code> , <code>open_source</code>                                                                        |
| <code>liteql/semantic.py</code>     | 82    | <code>analyze()</code> : column existence, type checking, LIMIT; returns the name-resolved AST                                                                                                                                              |
| <code>liteql/logical_plan.py</code> | 85    | <code>Scan</code> , <code>Filter</code> , <code>Project</code> , <code>Sort</code> , <code>Limit</code> and <code>build_plan()</code>                                                                                                       |

| File                                                          | Lines | What it does                                                                                                                                          |
|---------------------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>liteql/optimizer.py</code>                              | 101   | <code>optimize()</code> , <code>push_predicate</code> , <code>push_projection</code> , <code>required_columns</code> , <code>condition_columns</code> |
| <code>liteql/executor.py</code>                               | 82    | <code>execute()</code> , <code>evaluate()</code> for conditions, <code>ExecutionStats</code>                                                          |
| <code>liteql/errors.py</code>                                 | 58    | <code>LiteQLError</code> and one subclass per phase                                                                                                   |
| <code>liteql/compiler.py</code>                               | 73    | <code>compile_query()</code> chains the phases into a <code>Compilation</code> ; <code>run_query()</code>                                             |
| <code>liteql/printer.py</code>                                | 109   | AST tree, plan, token list and result-table formatting                                                                                                |
| <code>liteql/cli.py</code>                                    | 156   | Argument parsing, stage display, <code>--stats</code> , error markers, <code>--demo</code>                                                            |
| <code>liteql/__init__.py</code> ,<br><code>__main__.py</code> | 14    | Public exports; <code>python -m liteql</code> entry point                                                                                             |

## Everything else in the project

| Path                                                                          | Contents                                                                                                    |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>tests/</code>                                                           | <code>conftest.py</code> (shared fixtures) and ten test files, one per phase plus end-to-end and robustness |
| <code>examples/students.csv</code> , <code>students.json</code>               | The 8-row main dataset in both formats (name, age, city)                                                    |
| <code>examples/titanic.csv</code>                                             | 40-row synthetic Titanic-style sample with blank ages and names containing commas                           |
| <code>examples/valid_queries.txt</code> ,<br><code>invalid_queries.txt</code> | Ready-made queries; invalid ones tagged with the rejecting phase                                            |
| <code>examples/notes.txt</code>                                               | A <code>.txt</code> file used to demonstrate the unsupported-file-type error                                |
| <code>docs/</code>                                                            | <code>grammar.md</code> , <code>architecture.md</code> , <code>demo.md</code>                               |

| Path                                                                                    | Contents                                                                |
|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>README.md</code> , <code>pyproject.toml</code> ,<br><code>requirements.txt</code> | Overview, packaging, the single test dependency ( <code>pytest</code> ) |

**Dependencies between modules** flow one way down the pipeline: `compiler.py` imports every phase; the phases import `ast` , `schema` , `errors` and `tokens` , but never `compiler.py` or `cli.py` . The only module that opens data files is `data_source.py` (the executor and the driver (for the schema) reach files only through it); `cli.py` only prints.